Apprentissage par Renforcement sur Highway-v0

DQN Custom vs Stable-Baselines3

Groupe 17

Antoine Yezou, Maxence Rossignol, Zacharie Boumard, Ylias Larbi

Avril 2026

Sommaire

1. Environnement et Modélisation
2. DQN : Algorithme et Implémentation
3. Résultats et Comparaison
4. Extension et Conclusion

L'Environnement : Highway-v0

Le problème : conduire sur une autoroute à 4 voies parmi **45 véhicules** pendant 30 secondes, en maximisant la vitesse sans collision.

Configuration :

- 4 voies, 45 véhicules, durée 30s
- Vitesses cibles : $[20, 25, 30]$ m/s
- Récompense normalisée
- Collision $\Rightarrow$ fin d'épisode

5 actions discrètes :

1. Changer à gauche
2. Idle (maintenir)
3. Changer à droite
4. Accélérer
5. Ralentir

Actions de type `DiscreteMetaAction` *: un contrôleur bas niveau gère la trajectoire.*

État et Récompense

Observation (état s_t) : matrice cinématique 10×5

	présence	x	y	v_x	v_y
Ego	1.0	0.0	0.0	0.0	0.0
Voisin 1	1.0	-0.3	0.0	-0.2	0.0
Voisin 2	1.0	0.1	0.25	0.05	0.0
...	...	...	...	...	...

Valeurs **relatives** au véhicule ego, **normalisées** entre $[-1, 1]$.

Récompense à chaque pas :

- Vitesse élevée : $+0.7$ (progressif sur $[22, 30]$ m/s)
- Collision : -1.5 (fin d'épisode)
- Changement de voie : -0.02

Arbitrage

L'agent doit maximiser sa vitesse sans s'exposer au coût de collision ($\sim 2\times$ le bonus de vitesse).

DQN : Principe

Idée : apprendre une fonction $Q(s, a)$ qui prédit le **cumul de récompenses futures** pour chaque action.

Boucle d'apprentissage :

1. **Observer** l'état s_t (matrice 10×5 aplatie en vecteur de 50)
2. **Choisir** une action : aléatoire (probabilité ϵ) ou $\arg\max_a Q(s_t, a)$
3. **Exécuter** l'action, observer r_t, s_{t+1}
4. **Stocker** la transition $(s_t, a_t, r_t, s_{t+1}, done)$ dans le *replay buffer*
5. **Apprendre** sur un batch aléatoire de 64 transitions

Cible d'apprentissage :

$$y = r_t + \gamma \max_{a'} Q_{\text{target}}(s_{t+1}, a') \quad \text{avec } \gamma = 0.99$$

L'agent minimise $\mathcal{L} = (Q(s_t, a_t) - y)^2$ par descente de gradient.

Composants clés du DQN

Experience Replay

- Buffer circulaire de 50 000 transitions
- Apprentissage sur des transitions **aléatoires** (pas consécutives)
- Casse les corrélations temporelles $\Rightarrow$ stabilise l'entraînement

Exploration (ϵ-greedy)

- ϵ : 1.0 $\rightarrow$ 0.05 (linéaire)
- Décroissance sur 50% des timesteps
- 5% d'exploration résiduelle

Target Network

- Copie **gelée** du réseau
- Mise à jour tous les 500 pas
- Fournit des cibles stables pour éviter la divergence des Q-values

Réseau (MLP)

- Entrée : 50 (matrice aplatie)
- Couches : 256 $\rightarrow$ 128 (ReLU)
- Sortie : 5 Q-values
- Gradient clippé à 1.0

Protocole expérimental

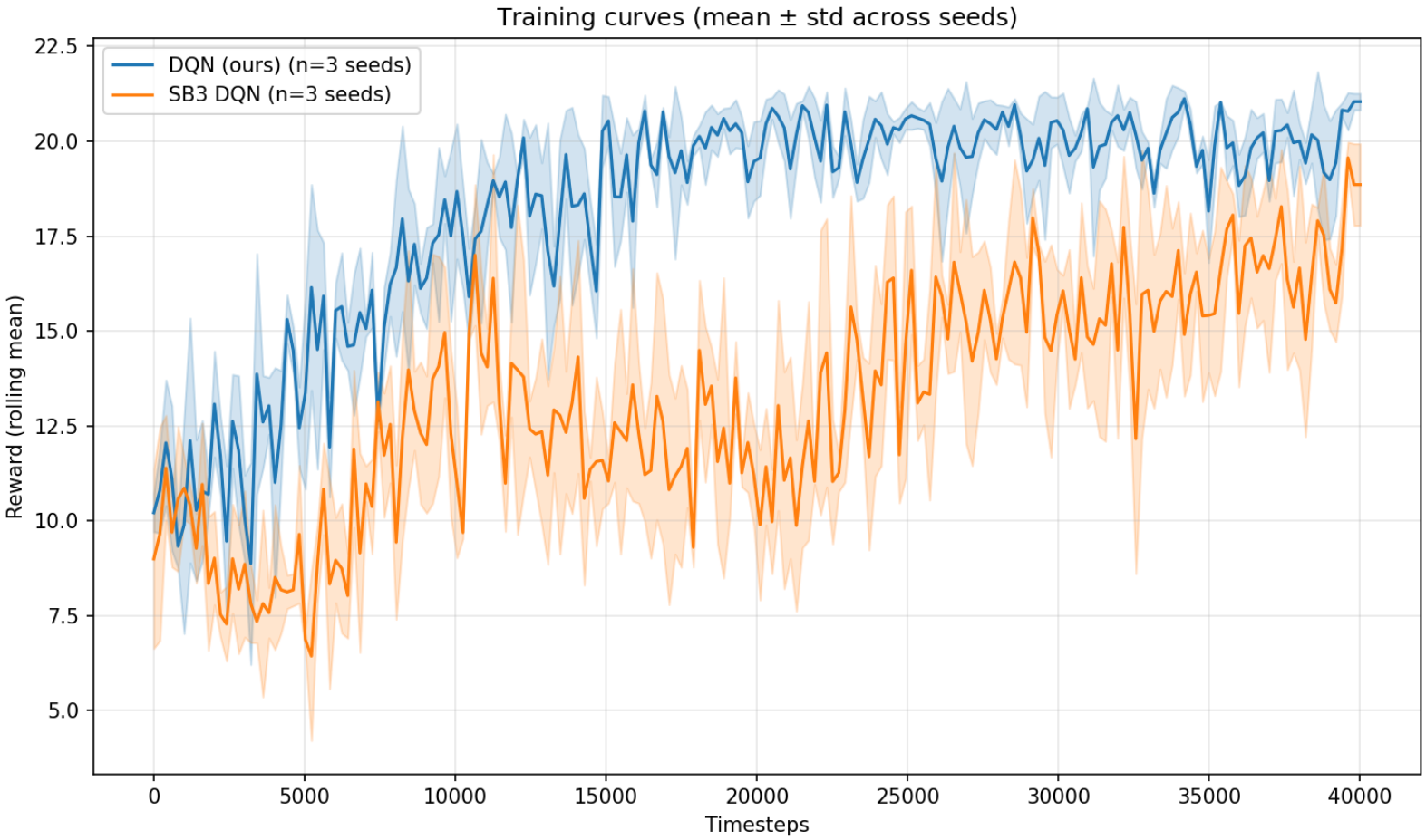
Comparaison équitable : même nombre de timesteps, même config, mêmes seeds.

	DQN (custom)	SB3 DQN
Timesteps	40 000	40 000
Seeds	42, 43, 44	42, 43, 44
Learning rate	10^{-3}	10^{-3}
Buffer	50 000	50 000
Batch size	64	64
Target update	500 steps	500 steps
ϵ schedule	$1 \to 0.05$ sur 50%	$1 \to 0.05$ sur 30%
Réseau	256-128 (custom)	`MlpPolicy` (SB3)

Évaluation : **50 épisodes** par seed, politique déterministe ($\epsilon = 0$).

Entraîné sur le cluster DCE de CentraleSupélec (NVIDIA GTX 1080 Ti, CUDA 12.4).

Courbes d'entraînement



Moyenne ± écart-type sur 3 seeds. Lissage par moyenne glissante (fenetre 10).

- Notre DQN converge en ∼5 000 timesteps. SB3 en ∼20 000.
- Variance inter-seeds beaucoup plus faible pour notre DQN.

Évaluation quantitative (50 runs × 3 seeds)

Agent	Seed	Reward moy.	Std	Crashes
DQN (custom)	42	20.68	2.70	6/50 (12%)
DQN (custom)	43	20.33	2.60	2/50 (4%)
DQN (custom)	44	20.53	2.29	4/50 (8%)
DQN (custom)	**total**	**20.51**	**2.54**	**12/150 (8%)**
SB3 DQN	42	19.52	3.47	13/50 (26%)
SB3 DQN	43	16.11	6.00	22/50 (44%)
SB3 DQN	44	10.77	7.35	42/50 (84%)
SB3 DQN	**total**	**15.47**	**6.85**	**77/150 (51%)**

Notre DQN : reward supérieure (+33%), variance plus faible, et **6× moins de crashes**.

Analyse des modes d'échec

Notre DQN (8% de crashes)

Deux patterns identifiés :

- **Crash précoce** (steps 3–9) : changement de voie dans une voie occupée juste après le reset.
- **Crash tardif** (steps 21–29) : collision en fin d'épisode lors d'un changement de voie dans du trafic rapide.

SB3 DQN (51% de crashes)

Modes d'échec plus variés :

- Crashes à haute vitesse (jusqu'à 28 m/s)
- Forte dépendance au seed : seed 44 $\Rightarrow$ **84% de crashes**
- Politique fragile : fonctionne sur certaines configs de trafic, pas d'autres

Hypothèse

SB3 souffre de son `learning_starts=1000` (pas d'apprentissage pendant les 1000 premiers pas) et d'un schedule d'exploration plus court (30% vs 50%).

Choix de design

- **Timesteps comme unité commune :** permet une comparaison directe DQN vs SB3 sur le même axe. L'ancienne version utilisait des épisodes (DQN) vs timesteps (SB3) $\Rightarrow$ incomparable.
- $\gamma = 0.99$ **:** horizon lointain nécessaire pour anticiper le trafic. Un γ plus faible rendrait l'agent trop myope (ne planifie pas les dépassements).
- **MLP 256-128 :** suffisant pour un espace d'état de dimension 50. Un réseau plus grand risquerait l'overfitting sans apporter de capacité de représentation utile.
- **Sauvegarde incrémentale (CSV) :** résultats enregistrés après chaque seed. A sauvé nos données quand le premier job cluster a été tué par SLURM.

Extension : *À compléter*

Direction envisagée : *[Ex : Double-DQN / Prioritized Experience Replay / Safety-RL]*

Hypothèse :

- *[Ex : Double-DQN réduit la surestimation des Q-values ⇒ politique plus stable]*

Protocole expérimental :

- *[Comparaison / ablation / variation contrôlée]*

Résultats :

- *[À compléter]*

Conclusion

Résultats principaux :

- Notre DQN custom **surpasse SB3** : reward moyenne 20.51 vs 15.47, taux de crash 8% vs 51%.
- Convergence plus rapide ($\sim$5k vs $\sim$20k timesteps) et plus reproductible (faible variance inter-seeds).

Leçons apprises :

- L'importance d'une comparaison équitable (même unité, même config).
- La robustesse de l'infrastructure (sauvegardes incrémentales, gestion du cluster).
- Les bugs silencieux (Monitor wrapper, observation space) peuvent invalider des résultats sans signal d'erreur.

Perspectives :

- Extension (Double-DQN, PER, Safety-RL)
- Tuning d'hyperparamètres (exploration schedule, architecture)
- Généralisation à d'autres environnements (intersection, rond-point)